

ASTRID

Autonomous Multi-Agent Browser AI & Cloud Automation Platform

Architecture & System Design Report

September 2026

Table of contents

Table of contents	2
1. Executive summary	4
1.1 Problem this system solves	4
1.2 What makes Astrid different	4
2. Capabilities overview	5
3. High-level system architecture	6
3.1 Client layer	6
3.2 Backend gateway	6
3.3 Multi-agent cognitive core	6
3.4 Tri-tier memory system	6
3.5 Hybrid execution engines	6
4. Multi-agent cognitive pipeline	8
4.1 Orchestrator agent	8
4.2 Planner agent	8
4.3 Verifier agent	8
4.4 Why separate these three roles	8
5. Execution subsystems	9
5.1 Live browser engine (Chrome DevTools Protocol)	9
5.2 Autonomous cloud engine (boto3)	9
5.3 Why two engines instead of one	9
6. Tri-tier memory architecture	10
6.1 Tier 1: in-flight plan memory	10
6.2 Tier 2: local session memory	10
6.3 Tier 3: global semantic vector memory	10
6.4 Why three tiers and not one	11
7. Web application architecture	12
7.1 Authentication and user data	12
7.2 Request lifecycle	12
7.3 Frontend responsibilities	12
7.4 Backend responsibilities	12
7.5 Interactive UI	12
8. End-to-end data flow	14
9. Technology stack	15
10. Deployment architecture	16
10.1 Environment isolation	16
11. Security considerations	17
12. Limitations and future work	18
13. Conclusion	19

Appendix A: Repository structure	20
Appendix B: Glossary	21

1. Executive summary

Astrid is an autonomous, multi-agent browser intelligence and cloud operations system built to carry out complex, multi-step web and cloud workflows on a user's behalf. Rather than relying on brittle, hardcoded automation scripts or a single large language model naively clicking through a page, Astrid separates reasoning into three cooperating agents (Orchestrator, Planner, Verifier), backed by a three-tier memory system, and routed through two specialized execution engines: a live browser controller and an autonomous cloud engine.

The system attaches to a real, already-authenticated Chrome session over the Chrome DevTools Protocol, preserving human logins and single sign-on state so it can act inside services like Canvas, Vocareum, and the AWS Console without triggering bot detection. For cloud objectives, Astrid extracts short-lived credentials from the active session and drives AWS directly through boto3, which is faster and far more reliable than simulating clicks through a cloud console.

This report documents the full system: the responsibilities of each agent, the tri-tier memory architecture, the dual execution engines, the end-to-end web application stack, the data flow for a representative task, the technology choices behind each layer, and the deployment and security model.

1.1 Problem this system solves

- Manual completion of long, repetitive browser-based workflows (lab exercises, quizzes, cloud provisioning tasks) is slow and error-prone.
- Naive LLM-driven browser agents burn tokens on pixel-level clicking and frequently hallucinate success without verifying it.
- Cloud infrastructure tasks performed by clicking through a web console are slow, hard to reproduce, and easy to get wrong.
- Repeated tasks (recurring quizzes, recurring lab setups) should not require re-solving the same problem from scratch every time.

1.2 What makes Astrid different

- Actor-critic separation: the agent that plans is not the agent that grades its own work.
- Dual execution: browser automation for UI-only surfaces, direct SDK calls for anything with an API.
- Persistent, semantic memory: past runs are embedded and retrieved for similar future goals, not thrown away.
- Live, authenticated browser control instead of a disposable, logged-out automation browser.

2. Capabilities overview

The table below summarizes Astrid's primary capabilities, how each is implemented, and the practical effect for the user.

Capability	Implementation	Impact
Live authenticated browser control	Attaches to an existing Chrome instance via CDP on port 9222	Operates inside real logins (Canvas, Vocareum, AWS SSO) without CAPTCHAs or re-authentication
Autonomous cloud provisioning	Session credential extraction + boto3	Provisions AMIs, ALBs, target groups, launch templates, auto scaling groups, and CloudWatch alarms directly
Cognitive quiz and exam solving	DOM traversal + Groq Llama 3.3 70B / Qwen 2.5 reasoning	Handles single-select, multi-select, and implied-count questions (e.g. "select three")
Tri-tier self-correcting memory	In-flight state + local JSON + Supabase pgvector	Never repeats a previously-identified mistake on a re-run
Real-time streaming interface	FastAPI WebSocket + React/Vite frontend	Live view of agent reasoning, actions, and progress as they happen

3. High-level system architecture

Astrid is organized into five layers: the client layer, the backend gateway, the multi-agent cognitive core, the tri-tier memory system, and the hybrid execution engines, which in turn act on two target environments.

3.1 Client layer

A React + Vite frontend renders the chat interface and a live subtask visualizer. It opens a single WebSocket connection per task and renders every event the backend streams: plan steps, actions taken, intermediate page state, and the final verified result.

3.2 Backend gateway

A FastAPI server exposes a WebSocket endpoint (/ws/task) guarded by Supabase JWT verification. Every inbound task carries the user's access token; the gateway resolves it to a user id before any agent work begins, so all memory reads and writes are scoped to that user.

3.3 Multi-agent cognitive core

Three agents cooperate in an actor-critic loop: the Orchestrator decomposes the goal and tracks progress, the Planner inspects page or resource state and decides the next concrete action, and the Verifier independently checks whether that action actually produced the required outcome before allowing the Orchestrator to advance.

3.4 Tri-tier memory system

Three memory tiers operate at different timescales: in-flight plan memory for the current run, a local JSON store for fast-turnaround corrections (such as quiz answers), and a global Supabase pgvector store for long-term, semantic recall across sessions and users.

3.5 Hybrid execution engines

Two engines carry out the Planner's decisions. The browser engine drives a real Chrome tab over the Chrome DevTools Protocol for anything that only exists as a UI. The cloud engine calls AWS directly through boto3 for anything with an API, which is both faster and more deterministic than clicking through a console.

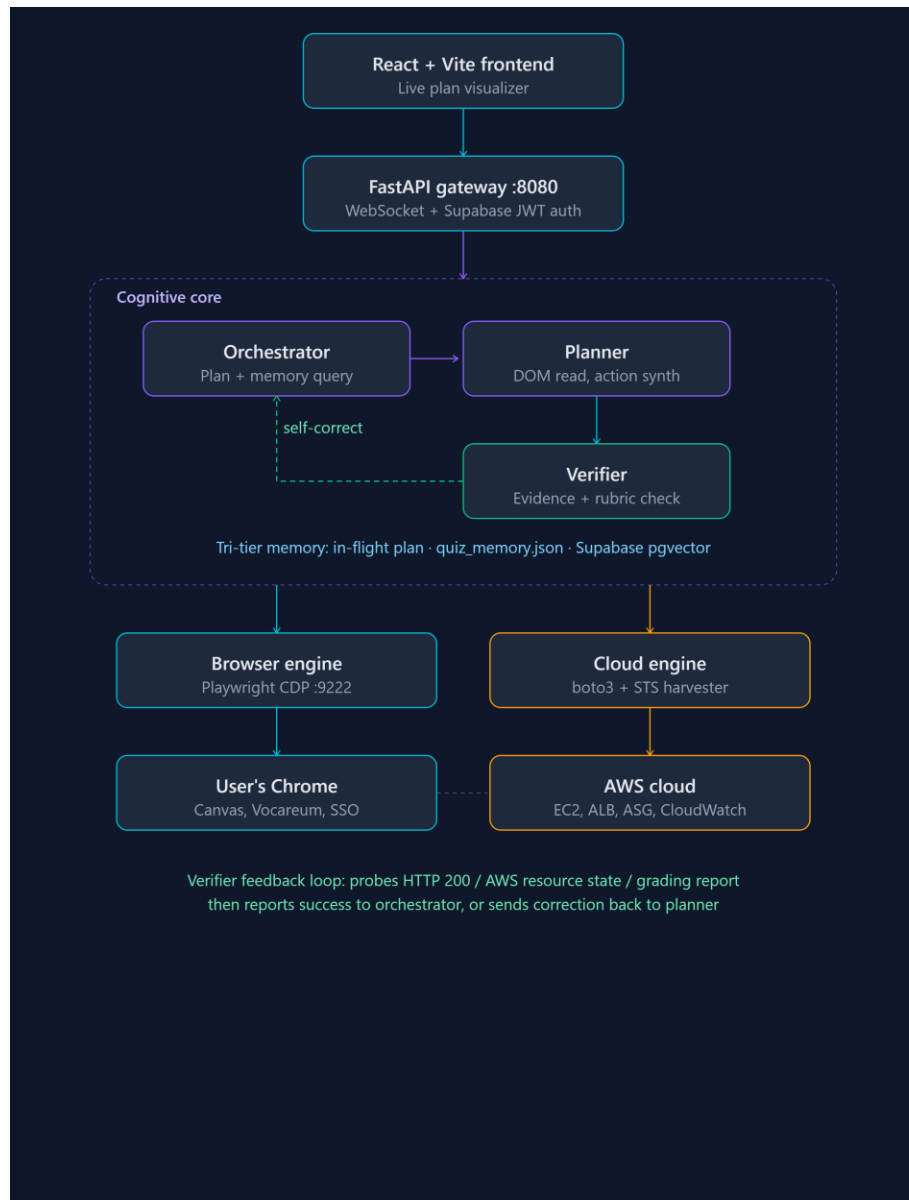


Figure 3.1 — Full system architecture: client, gateway, cognitive core, tri-tier memory, and the dual execution engines reaching their target environments.

4. Multi-agent cognitive pipeline

This section details each agent's responsibilities, inputs, outputs, and failure handling.

4.1 Orchestrator agent

Location: `backend/agents/orchestrator.py`

- Ingests the freeform user goal.
- Queries local and vector memory for similar past runs before planning, so known quirks or prior answers inform the plan.
- Produces a short, ordered list of atomic subtasks.
- Dispatches subtasks one at a time to the Planner and waits for Verifier confirmation before advancing.
- On completion, writes the run's outcome back into memory for future retrieval.

4.2 Planner agent

Location: `backend/agents/planner.py`

- Reads the current DOM, including nested iframes, to determine page state.
- Synthesizes the next concrete action: navigate, click, type, select an option, or invoke the cloud engine directly.
- Distinguishes single-select from multi-select questions, including cases where the required count is only implied by the question text.
- Routes cloud-shaped subtasks to the cloud engine instead of attempting to click through a web console, which is both slower and less reliable.

4.3 Verifier agent

Location: `backend/agents/verifier.py`

- Never accepts the Planner's own claim of success at face value.
- Probes independent evidence: HTTP status codes, AWS resource state via describe calls, or a parsed grading report.
- On success, confirms completion to the Orchestrator, which then advances the plan.
- On failure, returns a specific correction to the Planner, which retries with that feedback incorporated, rather than restarting the whole task.

4.4 Why separate these three roles

Collapsing planning and verification into one agent tends to produce optimistic self-grading: a model that decided an action was right is biased toward believing it worked. Keeping the Verifier structurally separate, and requiring it to check outside evidence rather than the Planner's own narration, is what allows Astrid to only report completion when the underlying task is actually done.

5. Execution subsystems

5.1 Live browser engine (Chrome DevTools Protocol)

The browser engine connects to an already-running, already-authenticated Chrome instance over CDP (default port 9222), rather than spawning a fresh, logged-out automation browser. This preserves cookies, session tokens, and SSO state, and avoids the bot-detection signals that a freshly-launched automation browser tends to trigger. It supports realistic mouse movement, smooth scrolling, keyboard input, and dismissal of modal overlays.

5.2 Autonomous cloud engine (boto3)

For AWS-shaped objectives, the cloud engine extracts temporary STS credentials directly from the active browser session (for example, from a Vocareum or AWS Academy lab environment) and initializes an authenticated boto3 session. From there it can execute a full infrastructure lifecycle:

- AMI lifecycle: locate a source instance, trigger image creation, and poll until the image is available.
- Load balancing: create a target group and an application load balancer with the appropriate listener.
- Scaling: configure a launch template and an auto scaling group with a target-tracking policy.
- Validation: confirm target health and that the load balancer responds with HTTP 200.
- Scale testing: raise desired capacity and confirm the expected number of healthy instances appear.
- Decommissioning: safely terminate instances that are no longer needed.
- Submission: submit the assessment and poll the grading report until it reflects full completion.

5.3 Why two engines instead of one

Routing anything with a stable API to the SDK path avoids spending LLM tokens and wall-clock time on fragile UI interactions, and avoids the class of failures where a UI element shifts position or a modal appears unexpectedly. The browser engine is reserved for surfaces that genuinely have no API, such as a quiz interface embedded in a learning management system.

6. Tri-tier memory architecture

Astrid's memory is deliberately split across three tiers with different lifetimes, so that fast, cheap corrections don't require a full semantic search, while long-term learning still accumulates across sessions.

6.1 Tier 1: in-flight plan memory

Held only in the backend process for the duration of one run. Tracks the active subtask index, the current plan, and a scratchpad of intermediate resource identifiers (an AMI id, a target group ARN, a load balancer DNS name) that later steps depend on.

6.2 Tier 2: local session memory

Persisted to a local `quiz_memory.json` file across restarts. Question hashes are matched against previously seen questions; if a question was ever answered incorrectly and later corrected, the correct option is loaded automatically on any future encounter, so the same mistake is never repeated.

6.3 Tier 3: global semantic vector memory

Supabase plays three distinct roles within this tier, not just one:

- Authentication: manages Google OAuth sign-in and issues the JWT that guards the `/ws/task` WebSocket gateway.
- Relational storage: persists user profiles, historical task runs, and step-by-step agent trajectories in PostgreSQL, scoped per user via Row-Level Security as defined in `backend/db/schema.sql`.
- Vector similarity search: stores 384-dimensional embeddings generated by all-MiniLM-L6-v2 in `pgvector`, and exposes a `match_memory()` function that semantically matches a new goal against previously successful execution trajectories.

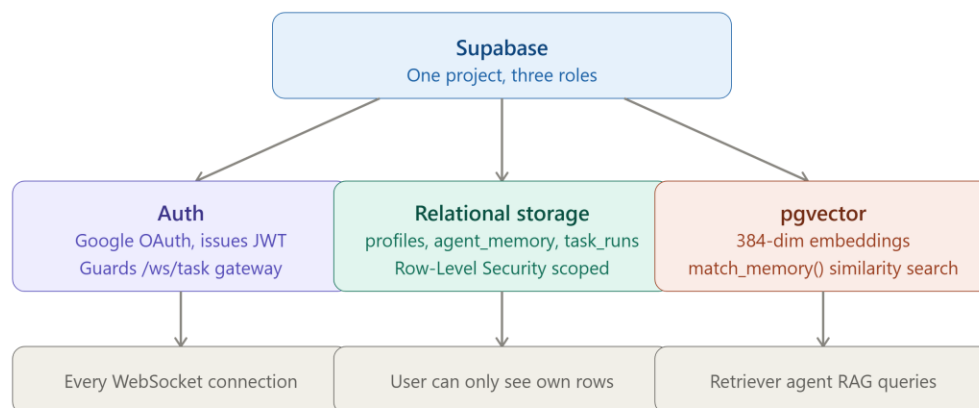


Figure 6.1 — Supabase's three distinct roles in Astrid: Auth guarding the gateway, RLS-scoped relational storage, and pgvector-backed semantic search.

6.4 Why three tiers and not one

- In-flight memory needs zero latency and no network round trip — it lives in process memory.
- Local session memory needs to survive a restart but doesn't need semantic search — an exact hash match is enough and is nearly instant.
- Global vector memory needs to generalize across differently-worded goals that mean the same thing, which requires embeddings and similarity search, and needs to be shared and durable, which is why it lives in Supabase rather than a local file.

7. Web application architecture

7.1 Authentication and user data

Users sign in with Google through Supabase Auth. Supabase issues a JWT that the frontend attaches to every WebSocket connection. On sign-up, a Postgres trigger automatically creates a profile row containing the user's name, email, and avatar URL, so no separate profile-management code is required in the backend.

7.2 Request lifecycle

- The user submits a goal in the chat interface.
- The frontend opens a WebSocket to the FastAPI gateway, carrying the Supabase access token.
- The gateway verifies the token, resolves the user id, and logs a new task run row.
- The Orchestrator, Planner, Verifier loop executes, streaming an event after every meaningful step.
- On completion, the final result and verification outcome are sent over the same socket, and the run's outcome is written back into vector memory.

7.3 Frontend responsibilities

The frontend is intentionally thin: it renders the conversation, renders a live plan/step visualizer driven entirely by server-sent events, and displays a live view of the browser or cloud engine's progress. It performs no independent reasoning.

7.4 Backend responsibilities

The backend owns all reasoning, memory access, and execution. It is stateless between WebSocket connections aside from what it reads from and writes to Supabase, which keeps it straightforward to scale horizontally on Cloud Run.

7.5 Interactive UI

The live interface shows the current instruction bar, a set of suggested tasks, and — directly under the input — a memory status strip: Orchestrator Memory, Planner Memory, and Verifier Memory as per-agent indicators, plus a separate Global Memory Active indicator. The per-agent indicators reflect Tier 1 in-flight state; Global Memory Active reflects Tier 3, the Supabase pgvector store, confirming that semantic recall from past runs is live for the current session. The right pane is a direct, click-through view of the live Chrome tab the browser engine is driving, with a manual override bar so a human can type or click directly into the page at any point.

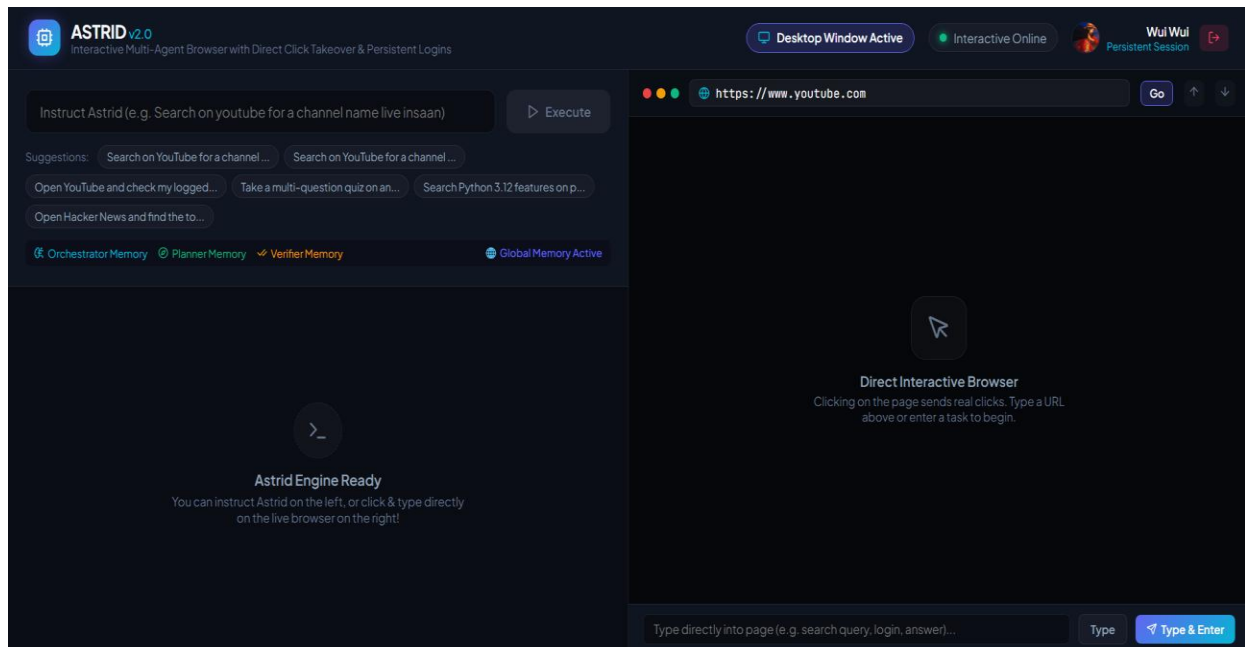


Figure 7.1 — Astrid v2.0 interface, showing per-agent memory indicators and Global Memory Active (Supabase pgvector) alongside the live interactive browser pane.

8. End-to-end data flow

A representative run, using an AWS lab objective as an example, proceeds as follows:

- 1. The user submits a high-level goal, for example completing a specific cloud infrastructure lab.
- 2. The Orchestrator queries memory for similar past runs, then generates an ordered subtask plan and streams it to the frontend.
- 3. For each subtask, the Planner inspects the current page or resource state.
- 4. The Planner decides whether the subtask belongs to the browser engine or the cloud engine, and dispatches accordingly.
- 5. The chosen engine executes the action and returns concrete evidence of its effect.
- 6. The Verifier independently checks that evidence against the subtask's success criteria.
- 7. On success, the Orchestrator advances to the next subtask and streams a progress update; on failure, the Verifier's specific correction goes back to the Planner for a retry.
- 8. Once every subtask is verified, the Orchestrator writes the outcome to memory and reports completion to the user.

9. Technology stack

Layer	Technologies
Frontend	React 18, Vite, Tailwind CSS, WebSocket client
Backend API	Python 3.12, FastAPI, Uvicorn, asyncio
Agent cognition	Groq API (Llama 3.3 70B versatile), Qwen 2.5, Pydantic
Browser control	Playwright for Python, Chrome DevTools Protocol
Cloud automation	AWS SDK (boto3)
Database and vector store	Supabase, PostgreSQL, pgvector, sentence-transformers
Security and auth	Supabase JWT, environment variable isolation

10. Deployment architecture

The backend is packaged as a container and deployed to Google Cloud Run, which scales to zero when idle and scales out under load without manual server management. The frontend is a static build deployed to Firebase Hosting or an equivalent static host. Supabase is used as a managed Postgres instance with pgvector and Auth enabled, removing the need to operate a separate database or authentication service.

10.1 Environment isolation

All credentials (Groq API key, Supabase service role key, AWS session tokens extracted at runtime) are supplied through environment variables and are never written to source control or logged in plaintext.

11. Security considerations

- All WebSocket connections are authenticated with a verified Supabase JWT before any agent work begins.
- Memory reads and writes are scoped per user id, so one user's task history is never retrievable by another — enforced at the Supabase Postgres layer via Row-Level Security (RLS) policies on profiles, agent_memory, and task_runs.
- AWS credentials extracted from a browser session are short-lived STS tokens, not long-lived access keys, limiting the blast radius of any leak.
- The system automates a browser and cloud account under the user's own authenticated session and control; it does not bypass authentication or authorization on any target system.

12. Limitations and future work

- The browser engine depends on a locally running Chrome instance with the remote debugging port open, which constrains deployment to environments where that can be arranged.
- The cloud engine currently targets AWS; extending it to other providers would require parallel credential-harvesting and SDK integration work.
- Verification is only as good as the probes it is given; new task types need an explicit, evidence-based check added before Astrid can be trusted to grade them.
- Future work includes broadening the tri-tier memory to support team-level sharing of learned corrections, and adding a fourth review step for higher-stakes cloud changes before they are applied.

13. Conclusion

Astrid demonstrates that a small number of well-separated agent roles, backed by tiered memory and routed through the right execution engine for each subtask, can complete genuinely complex, multi-step workflows without hardcoded scripts and without accepting a model's word for its own success. The architecture is deliberately modular: each agent, memory tier, and execution engine can be replaced or extended independently as new task types and target environments are added.

Appendix A: Repository structure

A representative layout of the Astrid codebase, grouped by layer.

- backend/agents/orchestrator.py — goal decomposition and plan tracking
- backend/agents/planner.py — DOM/state inspection and action synthesis
- backend/agents/verifier.py — independent evidence-based validation
- backend/agents/aws_lab_executor.py — autonomous cloud engine (boto3)
- backend/browser/agent.py — live browser engine (Playwright over CDP)
- backend/db/schema.sql — Supabase schema: profiles, agent_memory, match_memory(), task_runs
- backend/db/memory.py — embedding, storage, and retrieval helpers
- backend/routes/auth.py — Supabase JWT verification for REST and WebSocket routes
- backend/main.py — FastAPI app and /ws/task WebSocket handler
- frontend/src/ — chat interface, live plan visualizer, Supabase auth client

Appendix B: Glossary

Term	Meaning
CDP	Chrome DevTools Protocol, a WebSocket-based protocol for controlling and inspecting a real Chrome instance
STS	AWS Security Token Service, issues short-lived temporary credentials
pgvector	A PostgreSQL extension that adds a vector data type and similarity search operators
Actor-critic	A pattern where one component proposes actions and a separate component evaluates them
Embedding	A numeric vector representation of text used for semantic similarity search
Rubric	The set of scoring criteria a grading system checks a submission against